



Threads: Desvendando a Concorrência em Sistemas Operacionais

Aula 12 · Multithread · Threads do Kernel · Threads do Usuário · Modelos de Multithreading · Sincronização de Processos

O Que São Threads? A Essência da Concorrência

Uma **thread** (ou linha de execução) é a menor unidade de processamento que pode ser gerenciada de forma independente pelo sistema operacional. Diferente de processos, threads de um mesmo processo compartilham o mesmo espaço de endereçamento, tornando a comunicação entre elas muito mais eficiente e rápida.

Execução Paralela

Threads permitem que um único programa execute múltiplas tarefas ao mesmo tempo, como baixar um arquivo enquanto atualiza a interface gráfica.

Compartilhamento de Recursos

Threads do mesmo processo compartilham memória, arquivos abertos e outros recursos do processo pai, reduzindo a sobrecarga de comunicação.

Exploração de Multi-core

Em processadores com múltiplos núcleos, threads distintas podem ser executadas simultaneamente em núcleos diferentes, maximizando o desempenho computacional.

Responsividade

Aplicações multithreaded continuam respondendo ao usuário mesmo quando uma parte do programa está bloqueada em operações de I/O ou processamento intensivo.

Cada thread possui seu próprio **contador de programa**, conjunto de registradores e pilha de execução, mas compartilha com as demais threads do mesmo processo: o código, os dados globais e os recursos do sistema operacional. Isso torna a criação de threads consideravelmente mais barata do que a criação de novos processos.

Threads do Kernel vs. Threads do Usuário

Existem duas abordagens fundamentais para a implementação de threads, cada uma com suas características, vantagens e limitações. A escolha entre elas impacta diretamente o desempenho, a escalabilidade e o comportamento da aplicação frente ao sistema operacional.

Threads do Kernel

Gerenciadas diretamente pelo sistema operacional. O kernel conhece e escala cada thread individualmente.

- Suporte nativo a multiprocessadores e múltiplos núcleos
- Bloqueio de uma thread não afeta as demais do mesmo processo
- Maior overhead na criação e no chaveamento de contexto
- Exemplos: Windows NT/XP, Linux (NPTL), Solaris
- O kernel mantém informações de contexto para cada thread

Threads do Usuário

Gerenciadas por bibliotecas em espaço de usuário, sem conhecimento do kernel. O SO enxerga apenas um único processo.

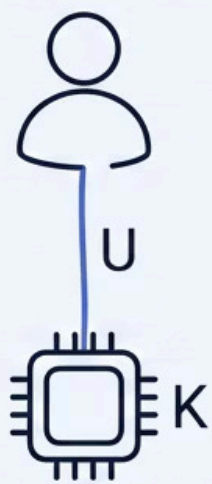
- Criação e chaveamento extremamente rápidos — sem chamada de sistema
- Portáveis entre diferentes sistemas operacionais
- Uma chamada de sistema bloqueante paralisa *todas* as threads
- Não aproveitam múltiplos núcleos de forma transparente
- Exemplos: LinuxThreads (antigo), GNU Pth, POSIX Pthreads

 Em sistemas modernos, a tendência é usar threads de kernel com bibliotecas que minimizam o overhead das chamadas de sistema.

Modelos de Multithreading: Como as Threads Trabalham Juntas

Os modelos de multithreading definem o **mapeamento entre threads do usuário e threads do kernel**. Cada modelo representa uma estratégia diferente para equilibrar desempenho, escalabilidade e compatibilidade com o sistema operacional subjacente.

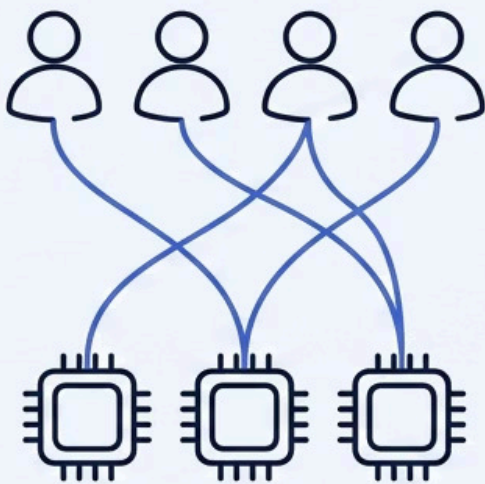
1	2	3
Modelo Um para Um (1:1) Cada thread criada pelo usuário tem uma correspondente thread de kernel. É o modelo mais simples e mais amplamente utilizado em sistemas modernos. • Vantagem: Paralelismo real em sistemas multiprocessados • Vantagem: Chamadas bloqueantes não afetam outras threads • Desvantagem: Overhead maior na criação — cada thread implica uma thread de kernel • Exemplos: Windows (desde NT), Linux com NPTL, macOS	Modelo Muitos para Um (N:1) Múltiplas threads de usuário são mapeadas para uma única thread de kernel. O gerenciamento é feito inteiramente em espaço de usuário. • Vantagem: Chaveamento de contexto extremamente eficiente • Desvantagem: Não explora paralelismo real em multi-core • Desvantagem: Uma chamada bloqueante paralisa todo o processo • Exemplos: LinuxThreads (implementação antiga), Green Threads (Java antigo)	Modelo Muitos para Muitos (M:N) Um número flexível de threads de usuário é mapeado para um número menor ou igual de threads de kernel. Combina as vantagens dos dois modelos anteriores. • Vantagem: Flexibilidade e alto desempenho • Vantagem: O kernel pode escalar threads conforme os núcleos disponíveis • Desvantagem: Implementação complexa • Exemplos: Solaris (versões antigas), HP-UX, IRIX



1) Um para Um



2) Muitos para Um



3) Muitos para Muitos

O Desafio da Concorrência: O Problema da Seção Crítica

Quando múltiplos threads acessam e manipulam dados compartilhados simultaneamente, sem nenhum mecanismo de controle, o resultado pode ser completamente imprevisível. Esse fenômeno é chamado de **condição de corrida** (*race condition*).

O Que é Seção Crítica?

A **seção crítica** é um trecho de código onde o processo acessa e modifica dados compartilhados. Apenas **um thread por vez** deve estar executando sua seção crítica para garantir a consistência dos dados.

Uma solução para o problema da seção crítica deve satisfazer três requisitos fundamentais:

- **Exclusão mútua:** Dois threads não podem estar simultaneamente em suas seções críticas
- **Progresso:** Threads fora da seção crítica não podem bloquear a entrada de outros
- **Espera limitada:** Um thread não deve esperar indefinidamente para entrar na seção crítica

Exemplo Clássico: Conta Bancária

Imagine dois threads tentando debitar e creditar o mesmo saldo simultaneamente:

```
Thread A lê saldo = R$1000
Thread B lê saldo = R$1000
Thread A adiciona R$500 → escreve R$1500
Thread B subtrai R$200 → escreve R$800
Saldo final: R$800 (INCORRETO!)
Deveria ser R$1300)
```

O resultado correto depende da ordem de execução — um comportamento inaceitável em sistemas críticos como bancos, controle de tráfego aéreo e sistemas médicos.

⊗ Condições de corrida são erros difíceis de reproduzir e depurar, pois dependem do escalonamento exato do sistema operacional.

Soluções de Sincronização: Hardware ao Resgate

O hardware moderno oferece mecanismos especiais de baixo nível que permitem implementar exclusão mútua de forma eficiente e confiável. Essas instruções são projetadas para ser **atômicas** — ou seja, são executadas em uma única etapa indivisível, sem possibilidade de interrupção pelo escalonador.



Test-and-Set (TAS)

Instrução atômica que lê o valor de uma variável e o define como verdadeiro em uma única operação indivisível. Usada para implementar **spinlocks** — travas onde o thread fica em espera ativa.

```
boolean test_and_set(boolean *lock) {  
    boolean rv = *lock;  
    *lock = TRUE;  
    return rv;  
}
```

Se o retorno for FALSE, o thread adquiriu a trava.
Se TRUE, o recurso está ocupado.



Compare-and-Swap (CAS)

Instrução atômica que compara o valor de uma variável com um valor esperado e, se iguais, o substitui por um novo valor. Amplamente usada em estruturas de dados *lock-free*.

```
int compare_and_swap(int *value,  
    int expected, int new_value) {  
    int temp = *value;  
    if (*value == expected)  
        *value = new_value;  
    return temp;  
}
```

Base de muitas implementações de mutex e semáforos em sistemas operacionais modernos.



Desabilitação de Interrupções

Em sistemas uniprocessados, desabilitar interrupções durante a seção crítica impede que o escalonador troque o contexto de execução. Simples, mas limitado a ambientes de kernel e sistemas de um único núcleo.

- Perigoso em modo usuário
- Ineficaz em multiprocessadores
- Usado em seções críticas do próprio kernel

 As instruções atômicas são implementadas diretamente em hardware com suporte do barramento de memória, garantindo que nenhum outro processador possa acessar a mesma posição de memória durante a execução da instrução.

Semáforos: Guardiões da Concorrência

Proposto por Edsger Dijkstra em 1965, o **semáforo** é uma das abstrações mais poderosas e elegantes da computação concorrente. É uma variável inteira especial cujo valor representa a quantidade de recursos disponíveis, e que só pode ser modificada por duas operações atômicas: **wait()** e **signal()**.

Operações Fundamentais

wait(S) — também chamada de **P()** ou *down()*:

```
wait(S) {  
    while (S <= 0)  
        ; // espera ativa (busy wait)  
    S--;  
}
```

signal(S) — também chamada de **V()** ou *up()*:

```
signal(S) {  
    S++;  
}
```

Ambas as operações são **indivisíveis** — nenhum outro processo pode executar uma operação no semáforo enquanto uma já estiver em execução.

Tipos de Semáforos

Semáforo Binário (Mutex): Assume apenas os valores 0 ou 1. Funciona como uma trava de exclusão mútua — apenas um thread por vez pode estar na seção crítica.

Semáforo Contador: Assume valores inteiros não-negativos. Controla o acesso a um recurso com múltiplas instâncias — por exemplo, um pool com N conexões de banco de dados disponíveis.

Semáforo com Fila de Espera

Para evitar a espera ativa (*busy waiting*), semáforos modernos bloqueiam o thread e o colocam em uma **fila de espera**. Quando o recurso é liberado via `signal()`, um thread da fila é acordado e movido para a fila de prontos.

- ✔ Semáforos implementados com bloqueio eliminam o desperdício de CPU causado por spinlocks, sendo essenciais para sistemas com alta concorrência.

Problemas Clássicos de Sincronização

Os problemas clássicos de sincronização são cenários canônicos usados para testar a correção e a eficiência de mecanismos de sincronização. Eles modelam situações reais encontradas em sistemas operacionais e aplicações distribuídas.

Problema Produtor-Consumidor (Buffer Limitado)

Um conjunto de threads **produtoras** gera itens e os insere em um buffer compartilhado de tamanho fixo. Um conjunto de threads **consumidoras** retira e processa esses itens. O desafio é garantir que:

- O produtor não insira itens em um buffer cheio
- O consumidor não retire itens de um buffer vazio
- Acesso ao buffer seja mutuamente exclusivo

Solução com semáforos: Usam-se três semáforos: **mutex** (exclusão mútua), **empty** (slots vazios) e **full** (slots ocupados). Exemplos reais: spooler de impressão, buffers de áudio/vídeo, filas de mensagens.

Problema dos Leitores e Escritores

Um banco de dados é compartilhado entre threads **leitoras** (que apenas consultam) e threads **escritoras** (que modificam os dados). As regras são:

- Múltiplos leitores podem acessar simultaneamente (sem conflito)
- Um escritor deve ter acesso exclusivo — nenhum outro leitor ou escritor pode estar ativo
- Dois escritores simultâneos causariam inconsistência

Existem variações que priorizam leitores (risco de inanição para escritores) ou escritores (risco de inanição para leitores). Exemplos reais: sistemas de banco de dados, caches compartilhadas, repositórios de configuração.

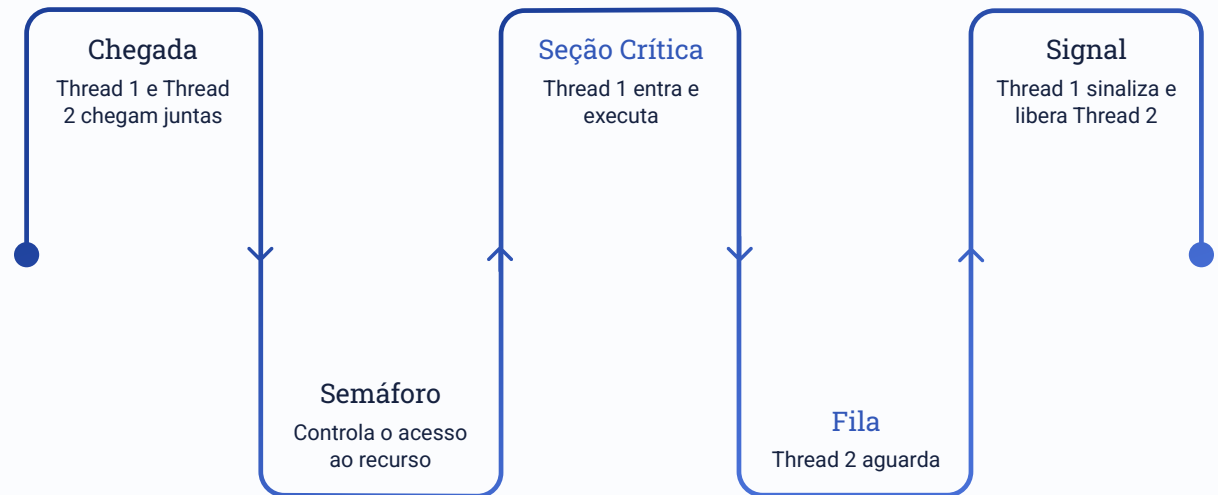
Problema do Jantar dos Filósofos

Cinco filósofos sentam em torno de uma mesa circular com cinco garfos. Para comer, cada filósofo precisa dos dois garfos adjacentes. Este problema modela cenários de **deadlock** e **inanição** — situações onde threads ficam bloqueadas indefinidamente esperando recursos que nunca serão liberados.

- Deadlock: todos pegam o garfo esquerdo e esperam o direito
- Solução: hierarquia de recursos, árbitro central, ou tentativa com timeout

Sincronização: A Chave para a Ordem no Caos Concorrente

Em sistemas concorrentes, a sincronização é o que separa um programa correto de um comportamento caótico e imprevisível.



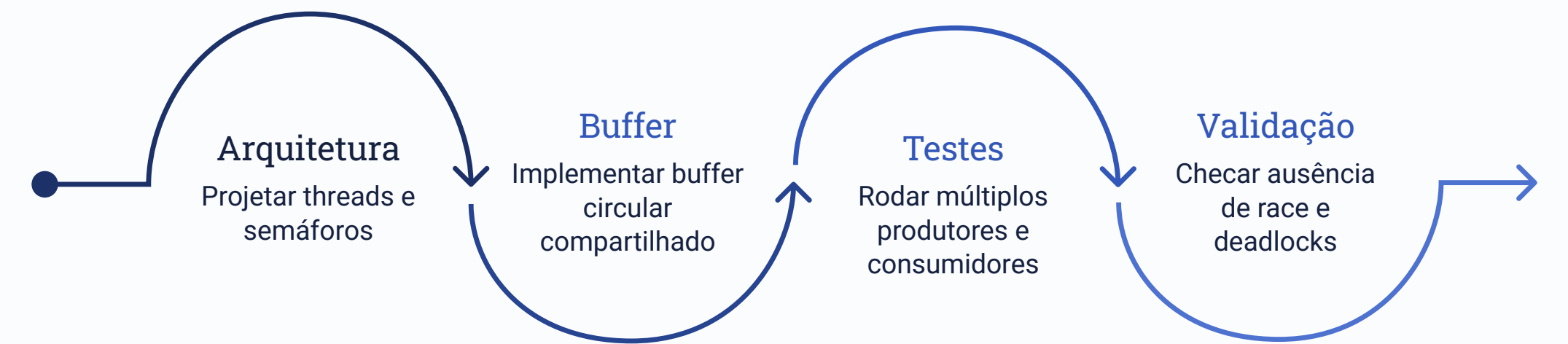
O semáforo atua como um **guardião** que garante ordem e previsibilidade em ambientes onde múltiplos threads competem pelos mesmos recursos. Sem ele, o sistema está sujeito a condições de corrida, deadlocks e resultados incorretos — problemas extremamente difíceis de diagnosticar em produção. Com a sincronização correta, é possível construir sistemas que escalam com segurança em ambientes multicore e distribuídos.

Desafio da Aula: Construindo um Sistema Concorrente Robusto

ATIVIDADE PRÁTICA

Aplique todos os conceitos estudados na aula para projetar e implementar um sistema concorrente completo. O objetivo é consolidar o entendimento de threads, modelos de multithreading e sincronização com semáforos em um cenário realista.

Cenário do Desafio Desenvolva um sistema com múltiplos produtores gerando dados (ex: leituras de sensores) e múltiplos consumidores processando esses dados (ex: salvando em banco), utilizando um buffer circular compartilhado de tamanho N.	Requisitos Técnicos • Implementar com Pthreads (C) ou Threads Java • Usar semáforos para controle de acesso ao buffer • Garantir exclusão mútua com mutex binário • Demonstrar ausência de deadlock e inanição	CrITÉrios de Avaliação • Corretude: os dados nunca são perdidos ou duplicados • Ausência de condições de corrida comprovada • Desempenho com 4+ produtores e 4+ consumidores • Documentação explicando as decisões de sincronização
--	---	--



❏ Dica: comece com 1 produtor e 1 consumidor, valide a correção, depois escale gradualmente para múltiplos threads. Utilize logs com timestamps para depurar a ordem de execução.